

Auckland University of Technology

**ENEL712 Project Report - Detailing
the Design, Development and
Functional Testing of a Laser
Condensation-Risk Simulator**

Aaron Taylor

A project report submitted to Auckland University of Technology
in partial fulfilment of the requirements for the degree of
Bachelor of Engineering (Honours)

June 2026

School of Engineering, Computer and Mathematical Sciences
Faculty of Design and Creative Technologies
Auckland University of Technology
New Zealand

Attestation of Authorship

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the qualification of any other degree or diploma of a university or other institution of higher learning.

Your Digital Signature Here

Signature of student

Abstract

Condensation on laser optics in non-climate-controlled New Zealand workshops can damage lenses and encourage faults. This report describes a laboratory-scale laser environment simulator on the AUT AT90USB1287 board with a Windows Forms supervisor.

Background. The project addresses dew-point and margin monitoring before enabling optical power, using the application board as a laser-head test rig rather than a production metal cutter.

Objectives. The system classifies condensation risk (Safe, Marginal, High), drives heater and fan PWM when the laser enable switch is active, and exposes readings to a PC GUI over UART at 38400 baud, with optional MySQL temperature logging.

Methodology. Firmware is brought up incrementally (`sim_piecel-sim_piece7`) and integrated in `final_project_firmware.c`; the GUI uses `IAppBoard`, `AppBoard`, and `SimulatorBoard` so development proceeds with or without hardware.

Results. Deliverables include integrated firmware (opcodes `0x00–0x0D` and `0x10–0x16`), a five-tab GUI, software simulator mode, and functional tests across Setup, Digital I/O, Ports-Lights, Temp Control, and Laser Sim tabs.

Conclusion. The work demonstrates a low-cost embedded supervisor for condensation-risk awareness and thermal behaviour representative of a water-cooled laser installation.

Contents

Attestation of Authorship	2
Abstract	3
Acknowledgements	8
Abbreviations	9
1 Introduction	10
1.1 Scope	10
1.2 Contributions	11
1.3 Report Structure	11
2 Project Design	12
2.1 Introduction	12
2.2 Project Proposal/Overview	12
2.2.1 Project Creation	12
2.3 Discussion	13
2.3.1 Challenges	13
2.3.2 Successes and Learnings	13
2.4 Chapter Summary	14
3 Project Development and Results	15
3.1 Introduction	15
3.2 OOP Class Structure	15
3.2.1 IAppBoard class Overview	16
3.2.2 AppBoard class Overview	17
3.2.3 SimulatorBoard class Overview	17
3.2.4 MainForm class Overview	17
3.2.5 DatabaseLogger class Overview	17
3.3 Microcontroller Code Structure	17
3.3.1 USART command dispatch Overview	17
3.3.2 Timer3 periodic tick Overview	18
3.3.3 Timer1 PWM and thermal loop Overview	18
3.4 Discussion	18

3.4.1	Challenges	18
3.4.2	Successes and Learnings	19
3.5	Chapter Summary	19
4	Conclusion	20
	Appendices	21
A	Yahtzee-Like Game Console Version - Class code	22
B	Yahtzee-Like Game GUI Version - Class code	33

List of Tables

List of Figures

3.1	System block diagram	16
-----	--------------------------------	----

Acknowledgements

I would like to express my gratitude to *the ENEL712 teaching team and project collaborators; specific acknowledgements to be completed before final submission.*

Abbreviations

SMD	Surface Mount Device.
ABR₁	Maths Abbreviation Example 1.
ABR...	Maths Abbreviation Example....
ABR_n	Maths Abbreviation Example n.
Ex₁	Example 1.
Ex₂	Example 2.
Ex_n	Example n.

Chapter 1

Introduction

This project report is submitted in partial fulfilment of the academic requirements for ENEL712 Semester One, 2026.

1.1 Scope

This Report includes a description of the process to design and implement the course project and labs. Specifically:

- i. The Project Itself - Laser Condensation-Risk Simulator**
- ii. Appendix 1 - Class Code for Yahtzee-Like Game Console Version**
- iii. Appendix 2 - Class Code for Yahtzee-Like Game GUI Version**

Each chapter provides relevant information that allows the reader to understand the project process from development to implementation. Such information includes descriptions of the design, development, results and conclusions.

1.2 Contributions

The persons contributing to this report are listed below:

Contributor 1 *Aaron Taylor (author) — firmware integration, PC GUI, simulator thermal model, and report drafting*

Contributor n *Iyla Blundell and Amber Joe (collaborators) — contributions to be marked in collaborator colour before submission*

1.3 Report Structure

The report is structured as follows:

Chapter 2 overviews the design process for the **Laser Condensation-Risk Simulator** Project. This chapter specifically highlights any challenges and learnings from this phase of the course.

Delete this magenta instruction before submission:

You may insert your project proposal or a modified version. The only content of this chapter that is marked is your discussion on challenges and learnings. The remainder of the chapter marks depend on whether or not you have populated the chapter or left it with the default text. **Default text = 0 marks**

Chapter 3 overviews the development and testing phases for the **Laser Condensation-Risk Simulator** Project. This chapter also discusses any challenges and learnings from this phase of the course.

Chapter 4 provides concluding remarks, and summarises the work carried out in this report, highlighting any key learnings from various stages of the project.

Chapter 2

Project Design

2.1 Introduction

This chapter includes the project [overview/proposal](#) and a discussion that includes any challenges or learnings in the design process for [the Laser Condensation-Risk Simulator](#).

2.2 Project [Proposal/Overview](#)

In this section the background to the project is presented.

2.2.1 [Project Creation](#)

Industrial laser systems in New Zealand workshops may operate without full climate control. Condensation on optics when surface temperature approaches dew point motivates monitoring humidity, head temperature, and margin before enabling laser power. The AUT AT90USB1287 board simulates the laser head: Pot 1 is laser power (heater PWM), humidity and temperature sensors feed a linear dew-point estimate $T_{\text{dew}} \approx T_{\text{ambient}} - (100 - \text{RH})/5$, and margin $M = T_{\text{surface}} - T_{\text{dew}}$ drives Safe ($M \geq 3^\circ\text{C}$),

Marginal ($0 \leq M < 3\text{ }^{\circ}\text{C}$), or High Risk ($M < 0\text{ }^{\circ}\text{C}$). SW0/SW1 encode phase; fan PWM tracks head temperature when the laser is enabled, with overtemperature heater cut-out. A Windows GUI polls opcodes 0x00–0x0D and extensions 0x10–0x16 at 38400 baud; optional MySQL logging supports the Temp Control tab.

2.3 Discussion

In this section the challenges and successes and learnings the author experienced during the design process are presented.

2.3.1 Challenges

This subsection lists the challenges experienced during the design process of the Project and how they were overcome.

- i. *Aligning the GUI opcode contract with Piece 7 firmware reads 0x10–0x16 while preserving baseline lab commands.*
- ii. *Reserving PORTC bits PC0–PC2 for risk LEDs while the Digital I/O tab still writes PC3–PC7 via SET_PORTC.*
- iii. *Testing without hardware using SimulatorBoard and the internal thermal model.*
- iv. *Partitioning Timer3 50 ms ticks (LED blink) from 1 Hz ADC and thermal updates in the main loop.*

2.3.2 Successes and Learnings

This subsection lists the highlights and learning the author experienced during the design process of the Project.

- i. *Incremental `sim_piecel-sim_piecel6` bring-up before integrated `final_project_final`*
- ii. *Shared `IAppBoard` interface for hardware and software simulator modes.*
- iii. *Retaining course baseline GUI tabs while adding the Laser Sim supervision tab.*

2.4 Chapter Summary

In this chapter, topics related to the design processes for the **Laser Condensation-Risk Simulator** Project were presented. In particular the following were highlighted: condensation-risk motivation, two-stage thermal and safety scope, UART protocol design, and design-phase challenges and learnings related to opcode alignment, PORTC sharing, and timer partitioning.

Chapter 3

Project Development and Results

3.1 Introduction

This chapter describes the development process undertaken to implement the **Laser Condensation-Risk Simulator** Project and the results obtained from functional testing of the system.

3.2 OOP Class Structure

This section details each of the classes implemented in the project, describing the functionality of each one.

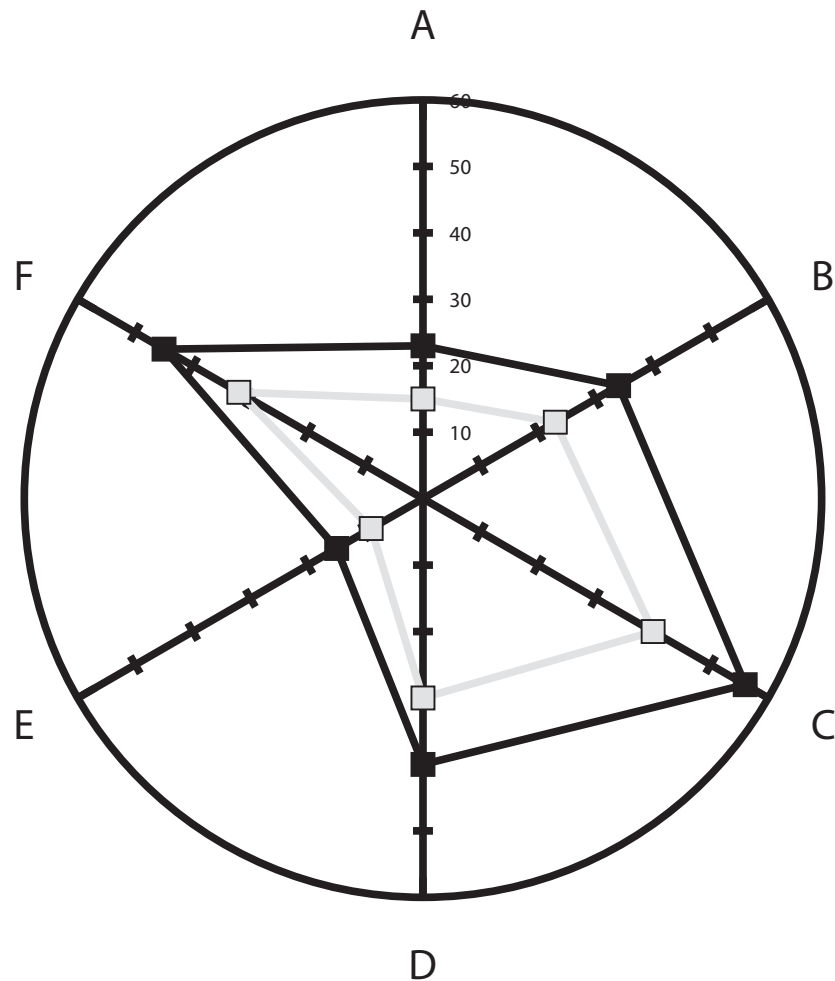


Figure 3.1: System block diagram — replace graphSample with project diagram from images-for-report/export before final submission if required

3.2.1 IAppBoard class Overview

Defines the common API for real hardware and the software simulator: connection, baseline reads and PWM writes, and laser-simulator methods (`ReadSimMargin`, `ReadSimStatus`, `ReadSimDewPoint`, `ReadSimPower`, `ReadSimFan`, `ReadAlarmFlags`).

3.2.2 **AppBoard class Overview**

Implements `IAppBoard` over `SerialPort` with 0x53/0xAA framing; `Ping()` uses `TXCHECK` (0x00) expecting reply 0x0F.

3.2.3 **SimulatorBoard class Overview**

Implements the same contract without hardware; the COM list entry `Simulator` selects an internal thermal model for all tabs.

3.2.4 **MainForm class Overview**

Hosts Setup, Digital I/O, Ports-Lights, Temp Control, and Laser Sim tabs; timer refresh, PI control, charting, and risk LEDs.

3.2.5 **DatabaseLogger class Overview**

Inserts temperature records into MySQL (`enel712_final`) when logging is enabled on the Temp Control tab.

3.3 **Microcontroller Code Structure**

This section details each of the significant functions implemented in the project, describing the purpose of each one.

3.3.1 **USART command dispatch Overview**

The `USART1_RX_ISR` parses framed packets; `main()` dispatches opcodes 0x00–0x0D and 0x10–0x16, encoding margin, status, dew point, power %, fan %, and alarm flags for the GUI.

3.3.2 Timer3 periodic tick Overview

A 50 ms compare-match ISR sets `g_tick_50ms`; the main loop performs 1 Hz ADC reads, recomputes dew point and margin, updates risk LED blink patterns, and drives the buzzer on High Risk.

3.3.3 Timer1 PWM and thermal loop Overview

Timer1 fast PWM ($ICR1 = 399$) drives fan, lamp, and heater OCR registers; the 1 Hz loop adjusts fan duty, maps Pot 1 to heater power, and enforces overtemperature lock-out with recovery timing.

3.4 Discussion

In this section the challenges and successes and learnings the author experienced during the design process are presented.

3.4.1 Challenges

This subsection lists the challenges experienced during the development process and functional testing of the Project and how they were overcome.

- i. *Autonomous firmware thermal updates overwriting host heater and fan writes during the same second; documented for testing and PI use on the Temp Control tab.*
- ii. *Matching C# decode of margin (+100 offset) and dew-point half-degree bytes to firmware encode_* helpers.*
- iii. *Validation without the physical board via SimulatorBoard and firmware-tests sources.*

3.4.2 Successes and Learnings

This subsection lists the highlights and learning the author experienced during the design process of the Project.

- i. *Integrated `final_project_firmware.c` combines UART service and the condensation or thermal background loop.*
- ii. *Functional testing confirms TXCHECK, Digital I/O, Ports-Lights, Temp Control (PI and chart), Laser Sim risk LEDs, and optional MySQL logging.*
- iii. *Lab 7 and Lab 8 Yahtzee OOP work is included in Appendices A and B as required by the report scope.*

3.5 Chapter Summary

In this chapter, topics related to the development and testing of the OOP C# code and microcontroller C code for the **Laser Condensation-Risk Simulator** project were presented, along with challenges and learnings experienced during the project implementation. In particular the following were highlighted: `IAppBoard/AppBoard/SimulatorBoard/MainForm` structure, firmware UART and timer modules, functional test results on all GUI tabs, and appendix listings for Lab 7 console and Lab 8 GUI class code.

Chapter 4

Conclusion

Delete Before Submission:

All text in this section to be written in past tense. It is a description of what was done and should make reference to where it can be seen in the report. Do not include any new information within this chapter.

This report examined the design, development and functional testing for the **Laser Condensation-Risk Simulator** project. In particular **Chapter 2** presented the project overview, dew-point and margin approach, and design-phase challenges and learnings; **Chapter 3** documented the C# and firmware structure, functional testing across all GUI tabs, and Lab 7/Lab 8 class code in the appendices; the integrated firmware and five-tab supervisor, including simulator mode, were shown to meet the ENEL712 final-project communication and supervision requirements.

Appendix A

Yahtzee-Like Game Console Version - Class code

Lab 7 console implementation (AaronYatzeeSimpleScoring), source: Labs/Lab 7/AaronYatzeeSimpleScoring/.

Program.cs

```
using System;
using System.Collections.Generic;

namespace AaronYatzeeSimpleScoring
{
    class Program
    {
        private const int DiceCount = 5;
        private const int RollsPerTurn = 3;
        private const int MaxPlayers = 5;

        static void Main(string[] args)
        {
```

```
Random rng = new Random();

Console.WriteLine("=== Yatzee ===");
Console.WriteLine("Each turn rolls 5 dice. Score = sum
↳ of dice.");
Console.WriteLine();

List<Player> players = ReadPlayers();
int rounds = ReadPositiveInt("How many rounds? ", min:
↳ 1);

for (int round = 1; round <= rounds; round++)
{
    foreach (Player player in players)
    {
        PlayOneTurn(player, rng, round, rounds);
    }
}

PrintFinalScoreboard(players);
}

private static void PlayOneTurn(Player player, Random rng,
↳ int round, int totalRounds)
{
    Console.WriteLine();
    Console.WriteLine($"--- Round {round}/{totalRounds},
↳ {player.Name}'s turn ---");

    List<Die> dice = NewDiceSet();

    // Roll 1: roll all dice.
```

```
RollAll(dice, rng);

DiceRowPrinter.PrintDiceRow(dice);

// Rolls 2 and 3: re-roll only the dice the player
↳ chooses.
for (int rollNumber = 2; rollNumber <= RollsPerTurn;
    ↳ rollNumber++)
{
    Console.WriteLine();
    List<int> indices = ReadRerollIndices();
    ReRoll(dice, rng, indices);
    DiceRowPrinter.PrintDiceRow(dice);
}

int turnScore = Scoring.SumDice(dice);
player.AddScore(turnScore);

Console.WriteLine();
Console.WriteLine($"{player.Name} scored {turnScore}
↳ this turn. Total: {player.TotalScore}");
Console.WriteLine("Press Enter for next turn...");
Console.ReadLine();
}

private static List<Die> NewDiceSet()
{
    List<Die> dice = new List<Die>();
    for (int i = 0; i < DiceCount; i++)
    {
        dice.Add(new Die());
    }
}
```



```
        return dice;
    }

    private static void RollAll(List<Die> dice, Random rng)
    {
        foreach (Die die in dice)
        {
            die.Roll(rng);
        }
    }

    private static void ReRoll(List<Die> dice, Random rng,
        ↪ List<int> indices)
    {
        foreach (int index in indices)
        {
            dice[index].Roll(rng);
        }
    }

    private static void PrintFinalScoreboard(List<Player>
        ↪ players)
    {
        Console.WriteLine();
        Console.WriteLine("=== Final scoreboard ===");
        foreach (Player p in players)
        {
            Console.WriteLine($"    {p.Name,-20}
                ↪ {p.TotalScore,5}");
        }

        Console.WriteLine();
    }
}
```

```
        Console.WriteLine("Thanks for playing!");
    }

    private static List<Player> ReadPlayers()
    {
        while (true)
        {
            Console.Write($"Number of players (1-{MaxPlayers})? ");
            string? line = Console.ReadLine();

            if (int.TryParse(line, out int count) && count >= 1
                && count <= MaxPlayers)
            {
                List<Player> players = new List<Player>();
                for (int i = 1; i <= count; i++)
                {
                    Console.Write($"Name for player {i}? ");
                    string? name = Console.ReadLine();

                    if (string.IsNullOrEmpty(name))
                    {
                        name = $"Player {i}";
                    }

                    players.Add(new Player(name.Trim()));
                }

                return players;
            }
        }
    }
}
```

```
        Console.WriteLine($"Please enter a number from 1 to
        ↪ {MaxPlayers}.");
    }
}

private static int ReadPositiveInt(string prompt, int min)
{
    while (true)
    {
        Console.Write(prompt);
        string? line = Console.ReadLine();

        if (int.TryParse(line, out int value) && value >=
        ↪ min)
        {
            return value;
        }

        Console.WriteLine($"Please enter a whole number >=
        ↪ {min}.");
    }
}

// Reads digits 1-5 to choose which dice to re-roll. Empty
↪ line keeps all.
private static List<int> ReadRerollIndices()
{
    while (true)
    {
        Console.Write("Dice to re-roll (e.g. 135), or Enter
        ↪ to keep all: ");
        string? line = Console.ReadLine();
```

```
if (string.IsNullOrEmpty(line))
{
    return new List<int>();
}

List<int> indices = new List<int>();
bool valid = true;

foreach (char c in line.Trim())
{
    if (c >= '1' && c <= '5')
    {
        int index = c - '1';
        if (!indices.Contains(index))
        {
            indices.Add(index);
        }
    }
    else
    {
        valid = false;
        break;
    }
}

if (valid)
{
    return indices;
}
```

```
        Console.WriteLine("Please use only digits 1-5 (e.g.  
        ↪ 24 or 135).");  
    }  
}  
}
```

Die.cs

```
using System;  
  
namespace AaronYatzeeSimpleScoring  
{  
    public class Die  
    {  
        public int Value { get; private set; }  
  
        public void Roll(Random rng)  
        {  
            Value = rng.Next(1, 7);  
        }  
    }  
}
```

DieFace.cs

```
namespace AaronYatzeeSimpleScoring  
{  
    public static class DieFace  
    {  
        public static string[] GetLines(int value)  
        {  
            switch (value)
```

```

        {
            case 1: return new[] { "+---+", "|   |", "| o |", "|
            ↪   |", "+---+" };
            case 2: return new[] { "+---+", "|o   |", "|   |", "|
            ↪   o|", "+---+" };
            case 3: return new[] { "+---+", "|o   |", "| o |", "|
            ↪   o|", "+---+" };
            case 4: return new[] { "+---+", "|o o|", "|   |",
            ↪   "|o o|", "+---+" };
            case 5: return new[] { "+---+", "|o o|", "| o |",
            ↪   "|o o|", "+---+" };
            case 6: return new[] { "+---+", "|o o|", "|o o|",
            ↪   "|o o|", "+---+" };
            default: return new[] { "?????", "?????", "?????",
            ↪   "?????", "?????" };
        }
    }
}

```

Player.cs

```

namespace AaronYatzeeSimpleScoring
{
    public class Player
    {
        public string Name { get; }
        public int TotalScore { get; private set; }

        public Player(string name)
        {
            Name = name;
        }
    }
}

```

```
        public void AddScore(int points)
        {
            TotalScore += points;
        }
    }
}
```

Scoring.cs

```
using System.Collections.Generic;

namespace AaronYatzeeSimpleScoring
{
    public static class Scoring
    {
        // Simple scoring rule: a turn scores the sum of all five
        // dice.
        public static int SumDice(List<Die> dice)
        {
            int total = 0;
            foreach (Die die in dice)
            {
                total += die.Value;
            }

            return total;
        }
    }
}
```

DiceRowPrinter.cs

```
using System;
```

```
using System.Collections.Generic;

namespace AaronYatzeeSimpleScoring
{
    public static class DiceRowPrinter
    {
        public static void PrintDiceRow(List<Die> dice)
        {
            const int LinesPerDie = 5;
            const string Gap = "    ";

            for (int line = 0; line < LinesPerDie; line++)
            {
                for (int i = 0; i < dice.Count; i++)
                {
                    string[] face = DieFace.GetLines(dice[i].Value);
                    Console.Write(face[line]);

                    if (i < dice.Count - 1)
                    {
                        Console.Write(Gap);
                    }
                }

                Console.WriteLine();
            }
        }
    }
}
```


Appendix B

Yahtzee-Like Game GUI Version - Class code

Lab 8 GUI implementation (YatzeeComplete), source: Labs/Lab8/YatzeeComplete/Yat

Program.cs

```
namespace YatzeeComplete
{
    internal static class Program
    {
        [STAThread]
        private static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new MainForm());
        }
    }
}
```

Die.cs

```
namespace YatzeeComplete
{
    public class Die
    {
        public int Value { get; private set; } = 1;

        // Default UNchecked: the player must tick the box to
        ↪ re-roll.
        public bool AllowRoll { get; set; } = false;

        private readonly Random _rng;

        public Die(Random rng)
        {
            _rng = rng;
        }

        // Re-rolls only when the player has marked the die.
        public void Roll()
        {
            if (AllowRoll)
            {
                Value = _rng.Next(1, 7);
            }
        }

        // Used for the very first roll of a turn (every die rolls
        ↪ regardless of ticks).
        public void ForceRoll()
        {
            Value = _rng.Next(1, 7);
        }
    }
}
```

```
    }  
}
```

DieFace.cs

```
using System.Drawing;  
  
namespace YatzeeComplete  
{  
    public static class DieFace  
    {  
        private static readonly Dictionary<int, Image> _cache =  
            ↪ new();  
  
        public static Image GetImage(int value)  
        {  
            if (_cache.TryGetValue(value, out Image? cached))  
            {  
                return cached;  
            }  
  
            string assetsDir =  
                ↪ Path.Combine(AppDomain.CurrentDomain.BaseDirectory,  
                ↪ "Assets");  
            string path = Path.Combine(assetsDir,  
                ↪ $"Die_{value}.png");  
  
            Image image;  
            if (File.Exists(path))  
            {  
                using Image temp = Image.FromFile(path);  
                image = new Bitmap(temp);  
            }  
        }  
    }  
}
```

```
        else
        {
            image = MakeFallback(value);
        }

        _cache[value] = image;
        return image;
    }

    private static Bitmap MakeFallback(int value)
    {
        Bitmap bmp = new Bitmap(120, 120);
        using Graphics g = Graphics.FromImage(bmp);
        g.Clear(Color.LightGoldenrodYellow);

        using Font font = new Font("Segoe UI", 36,
            ↪ FontStyle.Bold);
        string text = value.ToString();
        SizeF size = g.MeasureString(text, font);

        g.DrawString(text, font, Brushes.Black,
            (bmp.Width - size.Width) / 2,
            (bmp.Height - size.Height) / 2);
        g.DrawRectangle(Pens.DimGray, 0, 0, bmp.Width - 1,
            ↪ bmp.Height - 1);

        return bmp;
    }
}
```

Player.cs

```
namespace YatzeeComplete
{
    public class Player
    {
        public string Name { get; }
        public Scorecard Scorecard { get; } = new Scorecard();

        public Player(string name)
        {
            Name = name;
        }
    }
}
```

ScoreCategory.cs

```
namespace YatzeeComplete
{
    public enum ScoreCategory
    {
        Ones,
        Twos,
        Threes,
        Fours,
        Fives,
        Sixes,
        ThreeOfAKind,
        FourOfAKind,
        FullHouse,
        SmallStraight,
        LargeStraight,
        Yahtzee,
        Chance,
    }
}
```

```
}

public static class ScoreCategoryNames
{
    public static string DisplayName(ScoreCategory c) => c
        ↪ switch
        {
            ScoreCategory.Ones => "Ones",
            ScoreCategory.Twos => "Twos",
            ScoreCategory.Threes => "Threes",
            ScoreCategory.Fours => "Fours",
            ScoreCategory.Fives => "Fives",
            ScoreCategory.Sixes => "Sixes",
            ScoreCategory.ThreeOfAKind => "3 of a Kind",
            ScoreCategory.FourOfAKind => "4 of a Kind",
            ScoreCategory.FullHouse => "Full House",
            ScoreCategory.SmallStraight => "Small Straight",
            ScoreCategory.LargeStraight => "Large Straight",
            ScoreCategory.Yahtzee => "Yahtzee",
            ScoreCategory.Chance => "Chance",
            _ => c.ToString(),
        };
    }
}
```

Scorecard.cs

```
namespace YatzeeComplete
{
    public class Scorecard
    {
        private static readonly ScoreCategory[] AllCategories =
            (ScoreCategory[])Enum.GetValues(typeof(ScoreCategory));
    }
}
```

```
private readonly int?[] _scores = new
    ↪ int?[AllCategories.Length];

public bool IsFilled(ScoreCategory category) =>
    ↪ _scores[(int)category].HasValue;

public int? GetScore(ScoreCategory category) =>
    ↪ _scores[(int)category];

public bool IsComplete => AllCategories.All(IsFilled);

public void Record(ScoreCategory category, int points)
{
    int idx = (int)category;
    if (_scores[idx].HasValue)
    {
        throw new InvalidOperationException($"Category
            ↪ {category} is already filled.");
    }

    _scores[idx] = points;
}

public int UpperSum()
{
    int total = 0;
    for (int i = (int)ScoreCategory.Ones; i <=
        ↪ (int)ScoreCategory.Sixes; i++)
    {
        total += _scores[i] ?? 0;
    }
}
```

```
        return total;
    }

    /// <summary>+35 when the upper section reaches
    ↪ 63.</summary>
    public int UpperBonus => UpperSum() >= 63 ? 35 : 0;

    public int LowerSum()
    {
        int total = 0;
        for (int i = (int)ScoreCategory.ThreeOfAKind; i <=
            ↪ (int)ScoreCategory.Chance; i++)
        {
            total += _scores[i] ?? 0;
        }

        return total;
    }

    public int GrandTotal => UpperSum() + UpperBonus +
        ↪ LowerSum();

    public IEnumerable<ScoreCategory> UnfilledCategories()
    {
        foreach (ScoreCategory c in AllCategories)
        {
            if (!IsFilled(c))
            {
                yield return c;
            }
        }
    }
}
```



```
    }  
    }  
}
```

Scoring.cs

```
namespace YatzeeComplete  
{  
    public static class Scoring  
    {  
        /// <summary>  
        /// Scores the chosen category for the given five dice  
        /// → values.  
        /// Lower-section patterns return 0 when the roll does not  
        /// → satisfy them.  
        /// </summary>  
        public static int ScoreRoll(ScoreCategory category,  
            → IReadOnlyList<int> values)  
        {  
            int[] counts = new int[7];  
            int sum = 0;  
  
            foreach (int v in values)  
            {  
                if (v >= 1 && v <= 6)  
                {  
                    counts[v]++;  
                }  
  
                sum += v;  
            }  
  
            int maxSame = counts.Max();  
        }  
    }  
}
```

```
return category switch
{
    ScoreCategory.Ones => counts[1],
    ScoreCategory.Twos => counts[2] * 2,
    ScoreCategory.Threes => counts[3] * 3,
    ScoreCategory.Fours => counts[4] * 4,
    ScoreCategory.Fives => counts[5] * 5,
    ScoreCategory.Sixes => counts[6] * 6,
    ScoreCategory.ThreeOfAKind => maxSame >= 3 ? sum :
        ↪ 0,
    ScoreCategory.FourOfAKind => maxSame >= 4 ? sum : 0,
    ScoreCategory.FullHouse => IsFullHouse(counts) ? 25 :
        ↪ 0,
    ScoreCategory.SmallStraight => HasStraight(counts,
        ↪ 4) ? 30 : 0,
    ScoreCategory.LargeStraight => HasStraight(counts,
        ↪ 5) ? 40 : 0,
    ScoreCategory.Yahtzee => maxSame == 5 ? 50 : 0,
    ScoreCategory.Chance => sum,
    _ => 0,
};
}

private static bool IsFullHouse(int[] counts)
{
    bool hasThree = false;
    bool hasTwo = false;

    for (int i = 1; i <= 6; i++)
    {
        if (counts[i] == 3)
```

```
        {
            hasThree = true;
        }
        else if (counts[i] == 2)
        {
            hasTwo = true;
        }
    }

    return hasThree && hasTwo;
}

private static bool HasStraight(int[] counts, int length)
{
    int run = 0;
    for (int i = 1; i <= 6; i++)
    {
        run = counts[i] > 0 ? run + 1 : 0;
        if (run >= length)
        {
            return true;
        }
    }

    return false;
}
}
```

Prompt.cs

```
namespace YatzeeComplete
{
```

```
/// <summary>
/// Tiny modal input dialog used during player setup.
/// Keeps the project WinForms-only (no extra packages).
/// </summary>
public static class Prompt
{
    public static string? Ask(string title, string message,
        ↪ string defaultValue = "")
    {
        using var form = new Form
        {
            Width = 380,
            Height = 170,
            Text = title,
            FormBorderStyle = FormBorderStyle.FixedDialog,
            StartPosition = FormStartPosition.CenterParent,
            MaximizeBox = false,
            MinimizeBox = false,
        };

        var lbl = new Label
        {
            Left = 12,
            Top = 12,
            AutoSize = true,
            Text = message,
        };

        var tb = new TextBox
        {
            Left = 12,
            Top = 40,
```

```
        Width = 340,
        Text = defaultValue,
    };

    var ok = new Button
    {
        Text = "OK",
        Left = 200,
        Top = 80,
        Width = 70,
        DialogResult = DialogResult.OK,
    };

    var cancel = new Button
    {
        Text = "Cancel",
        Left = 280,
        Top = 80,
        Width = 70,
        DialogResult = DialogResult.Cancel,
    };

    form.Controls.Add(lbl);
    form.Controls.Add(tb);
    form.Controls.Add(ok);
    form.Controls.Add(cancel);
    form.AcceptButton = ok;
    form.CancelButton = cancel;
    tb.SelectAll();

    DialogResult result = form.ShowDialog();
    return result == DialogResult.OK ? tb.Text : null;
```

```
}

public static int AskInt(string title, string message, int
↳ min, int max, int defaultValue)
{
    while (true)
    {
        string? answer = Ask(title, $"{message}
↳ ({min}-{max})", defaultValue.ToString());

        // Treat Cancel as "use default" so the user is
        ↳ never stuck.
        if (answer == null)
        {
            return defaultValue;
        }

        if (int.TryParse(answer.Trim(), out int n) && n >=
↳ min && n <= max)
        {
            return n;
        }

        MessageBox.Show(
            $"Please enter an integer between {min} and
↳ {max}.",
            title,
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);
    }
}
}
```

```
}
```

MainForm.cs

```
using System.Text;

namespace YatzeeComplete
{
    public sealed class MainForm : Form
    {
        private const int DiceCount = 5;
        private const int MaxRollsPerTurn = 3;

        private readonly Random _rng = new Random();
        private readonly List<Die> _dice = new List<Die>();
        private readonly PictureBox[] _dicePbs = new
            ↳ PictureBox[DiceCount];
        private readonly CheckBox[] _diceChecks = new
            ↳ CheckBox[DiceCount];

        /* Game state */
        private readonly List<Player> _players = new List<Player>();
        private int _currentPlayerIndex = 0;
        private int _rollsThisTurn = 0;
        private bool _gameActive = false;

        /* UI controls */
        private readonly Label _lblHeader = new Label();
        private readonly Button _btnRoll = new Button();
        private readonly Label _lblRollHint = new Label();
        private readonly Label _lblCategoriesHeader = new Label();
        private readonly ListBox _lbCategories = new ListBox();
        private readonly Button _btnScore = new Button();
    }
}
```

```
private readonly Label _lblScoreboardHeader = new Label();
private readonly RichTextBox _rtbScoreboard = new
    ↳ RichTextBox();
private readonly Button _btnNewGame = new Button();
private readonly Label _lblStatus = new Label();

public MainForm()
{
    Text = "Yahtzee Complete";
    StartPosition = FormStartPosition.CenterScreen;
    ClientSize = new Size(740, 600);
    MinimumSize = new Size(740, 600);

    for (int i = 0; i < DiceCount; i++)
    {
        _dice.Add(new Die(_rng));
    }

    BuildUi();
    ShowIdleUi();

    // Auto-prompt for player setup as soon as the window is
    ↳ shown.
    Shown += (s, e) => StartNewGame();
}

private void BuildUi()
{
    const int leftX = 20;

    // Header.
    _lblHeader.Location = new Point(leftX, 15);
```



```
_lblHeader.AutoSize = true;
_lblHeader.Font = new Font(Font.FontFamily, 11,
    ↳ FontStyle.Bold);
Controls.Add(_lblHeader);

// Dice strip + per-die "Roll" checkboxes.
const int dieSize = 80;
const int spacing = 15;
const int diceY = 50;
int diceX = leftX;

for (int i = 0; i < DiceCount; i++)
{
    int dieIndex = i;

    var pb = new PictureBox
    {
        Size = new Size(dieSize, dieSize),
        Location = new Point(diceX, diceY),
        BorderStyle = BorderStyle.FixedSingle,
        BackColor = Color.LightGray,
        SizeMode = PictureBoxSizeMode.Zoom,
    };
    Controls.Add(pb);
    _dicePbs[i] = pb;

    var cb = new CheckBox
    {
        Text = "Roll",
        Checked = false,
        AutoSize = true,
```

```
        Location = new Point(diceX + (dieSize - 50) / 2,
                               ↪ diceY + dieSize + 6),
    };

    cb.CheckedChanged += (s, e) =>
        ↪ _dice[dieIndex].AllowRoll = cb.Checked;
    Controls.Add(cb);
    _diceChecks[i] = cb;

    diceX += dieSize + spacing;
}

// Roll button + helper text beside it.
_btnRoll.Text = "Roll";
_btnRoll.Location = new Point(leftX, 180);
_btnRoll.Size = new Size(110, 30);
_btnRoll.Click += OnRollClicked;
Controls.Add(_btnRoll);

_lblRollHint.Location = new Point(leftX + 130, 187);
_lblRollHint.AutoSize = true;
_lblRollHint.ForeColor = Color.DimGray;
Controls.Add(_lblRollHint);

// Categories list (left).
_lblCategoriesHeader.Text = "Choose a category to
    ↪ score:";
_lblCategoriesHeader.Location = new Point(leftX, 230);
_lblCategoriesHeader.AutoSize = true;
Controls.Add(_lblCategoriesHeader);

_lbCategories.Location = new Point(leftX, 250);
_lbCategories.Size = new Size(330, 270);
```

```
_lbCategories.Font = new Font("Consolas", 10);
Controls.Add(_lbCategories);

_btnScore.Text = "Score Selected";
_btnScore.Location = new Point(leftX, 530);
_btnScore.Size = new Size(160, 35);
_btnScore.Click += OnScoreClicked;
Controls.Add(_btnScore);

// Scoreboard (right).
_lblScoreboardHeader.Text = "Scoreboard:";
_lblScoreboardHeader.Location = new Point(390, 230);
_lblScoreboardHeader.AutoSize = true;
Controls.Add(_lblScoreboardHeader);

_rtbScoreboard.Location = new Point(390, 250);
_rtbScoreboard.Size = new Size(330, 270);
_rtbScoreboard.ReadOnly = true;
_rtbScoreboard.Font = new Font("Consolas", 10);
_rtbScoreboard.BackColor = SystemColors.Window;
Controls.Add(_rtbScoreboard);

_btnNewGame.Text = "New Game";
_btnNewGame.Location = new Point(580, 530);
_btnNewGame.Size = new Size(140, 35);
_btnNewGame.Click += (s, e) => StartNewGame();
Controls.Add(_btnNewGame);

// Status line at the bottom.
_lblStatus.Location = new Point(200, 540);
_lblStatus.AutoSize = true;
_lblStatus.ForeColor = Color.DimGray;
```

```
        Controls.Add(_lblStatus);
    }

    /// <summary>Runs once at startup before the game has
    ↪ started.</summary>
    private void ShowIdleUi()
    {
        _lblHeader.Text = "Press 'New Game' to start.";
        _btnRoll.Enabled = false;
        _btnScore.Enabled = false;
        _lblRollHint.Text = string.Empty;
    }

    private void StartNewGame()
    {
        int playerCount = Prompt.AskInt(
            "New Game",
            "How many players?",
            min: 1, max: 5, defaultValue: 1);

        _players.Clear();
        for (int i = 1; i <= playerCount; i++)
        {
            string fallback = $"Player {i}";
            string? name = Prompt.Ask(
                $"Player {i}",
                $"Enter name for player {i}:",
                fallback);

            if (string.IsNullOrEmpty(name))
            {
                name = fallback;
            }
        }
    }
}
```

```
    }

    _players.Add(new Player(name.Trim()));
}

_currentPlayerIndex = 0;
_rollsThisTurn = 0;
_gameActive = true;

ResetDiceForNewTurn();
_lblStatus.Text = string.Empty;
UpdateAll();
}

private void OnRollClicked(object? sender, EventArgs e)
{
    if (!_gameActive || _rollsThisTurn >= MaxRollsPerTurn)
    {
        return;
    }

    if (_rollsThisTurn == 0)
    {
        // First roll of a turn: every die is rolled
        ↪ regardless of ticks.
        for (int i = 0; i < DiceCount; i++)
        {
            _dice[i].ForceRoll();
            _dicePbs[i].Image =
                ↪ DieFace.GetImage(_dice[i].Value);
        }
    }
}
```

```
else
{
    // Subsequent rolls: only re-roll dice the player
    ↪ has ticked.
    bool anyTicked = false;
    for (int i = 0; i < DiceCount; i++)
    {
        if (_dice[i].AllowRoll)
        {
            _dice[i].Roll();
            _dicePbs[i].Image =
                ↪ DieFace.GetImage(_dice[i].Value);
            anyTicked = true;
        }
    }

    if (!anyTicked)
    {
        _lblStatus.Text = "Tick at least one die to
            ↪ re-roll, or score the current roll.";
        return;
    }
}

_rollsThisTurn++;
_lblStatus.Text = string.Empty;
UpdateAll();
}

private void OnScoreClicked(object? sender, EventArgs e)
{
    if (!_gameActive)
```

```
{
    return;
}

if (_rollsThisTurn == 0)
{
    _lblStatus.Text = "Roll at least once before
        ↳ scoring.";
    return;
}

int sel = _lbCategories.SelectedIndex;
if (sel < 0)
{
    _lblStatus.Text = "Pick a category from the list
        ↳ first.";
    return;
}

Player current = _players[_currentPlayerIndex];
List<ScoreCategory> unfilled =
    ↳ current.Scorecard.UnfilledCategories().ToList();
if (sel >= unfilled.Count)
{
    return;
}

ScoreCategory chosen = unfilled[sel];
int[] values = _dice.Select(d => d.Value).ToArray();
int points = Scoring.ScoreRoll(chosen, values);

current.Scorecard.Record(chosen, points);
```

```
        _lblStatus.Text =  
            $"{current.Name} scored {points} in  
            ↳ {ScoreCategoryNames.DisplayName(chosen)}.";   
  
        // Advance to next player; wrap to first when everyone  
        ↳ has played this round.  
        _currentPlayerIndex = (_currentPlayerIndex + 1) %  
            ↳ _players.Count;  
        _rollsThisTurn = 0;  
        ResetDiceForNewTurn();   
  
        // Game ends when every player has filled all 13  
        ↳ categories.  
        if (_players.All(p => p.Scorecard.IsComplete))  
        {  
            EndGame();  
            return;  
        }   
  
        UpdateAll();  
    }  
  
    private void EndGame()  
    {  
        _gameActive = false;  
        _btnRoll.Enabled = false;  
        _btnScore.Enabled = false;  
        _lblHeader.Text = "Game over!";  
        _lblRollHint.Text = string.Empty;  
        _lbCategories.Items.Clear();  
  
        UpdateScoreboard();  
    }  
}
```



```
List<Player> sorted = _players
    .OrderByDescending(p => p.Scorecard.GrandTotal)
    .ToList();

var sb = new StringBuilder();
sb.AppendLine("Final scores:");
for (int i = 0; i < sorted.Count; i++)
{
    sb.AppendLine($"    {i + 1}. {sorted[i].Name} --
        ↳ {sorted[i].Scorecard.GrandTotal}");
}

sb.AppendLine();
sb.AppendLine($"Winner: {sorted[0].Name}!");
sb.AppendLine();
sb.AppendLine("Press 'New Game' to play again.");

MessageBox.Show(
    sb.ToString(),
    "Game over",
    MessageBoxButtons.OK,
    MessageBoxIcon.Information);
}

private void ResetDiceForNewTurn()
{
    for (int i = 0; i < DiceCount; i++)
    {
        _dicePbs[i].Image = null;
        _diceChecks[i].Checked = false;
    }
}
```

```
}

private void UpdateAll()
{
    UpdateHeader();
    UpdateRollControls();
    UpdateCategoriesList();
    UpdateScoreboard();
}

private void UpdateHeader()
{
    if (!_gameActive)
    {
        _lblHeader.Text = "Press 'New Game' to start.";
        return;
    }

    Player current = _players[_currentPlayerIndex];
    int displayedRoll = Math.Min(_rollsThisTurn + 1,
        ↪ MaxRollsPerTurn);

    _lblHeader.Text =
        $"{current.Name}'s turn -- Roll
        ↪ {displayedRoll}/{MaxRollsPerTurn}";
}

private void UpdateRollControls()
{
    _btnRoll.Enabled = _gameActive && _rollsThisTurn <
        ↪ MaxRollsPerTurn;
}
```

```
        if (!_gameActive)
        {
            _lblRollHint.Text = string.Empty;
            return;
        }

        _lblRollHint.Text = _rollsThisTurn switch
        {
            0 => "First roll: rolls all five dice.",
            1 or 2 => "Tick the dice you want to re-roll.",
            _ => "No rolls left -- pick a category to score.",
        };
    }

    private void UpdateCategoriesList()
    {
        _lbCategories.Items.Clear();

        if (!_gameActive)
        {
            _btnScore.Enabled = false;
            return;
        }

        Player current = _players[_currentPlayerIndex];
        int[] values = _dice.Select(d => d.Value).ToArray();

        foreach (ScoreCategory cat in
            ↪ current.Scorecard.UnfilledCategories())
        {
            int preview = _rollsThisTurn == 0
                ? 0
```

```
        : Scoring.ScoreRoll(cat, values);

    string name = ScoreCategoryNames.DisplayName(cat);
    _lbCategories.Items.Add($"{name,-16} {preview,4}
        ↪ pts");
}

_btnScore.Enabled =
    _gameActive && _rollsThisTurn > 0 &&
    ↪ _lbCategories.Items.Count > 0;
}

private void UpdateScoreboard()
{
    var sb = new StringBuilder();

    for (int i = 0; i < _players.Count; i++)
    {
        Player p = _players[i];
        bool isCurrent = _gameActive && i ==
            ↪ _currentPlayerIndex;
        string marker = isCurrent ? "> " : " ";

        int filled = 13 -
            ↪ p.Scorecard.UnfilledCategories().Count();

        sb.AppendLine($"{marker}{p.Name}");
        sb.AppendLine($"    Upper: {p.Scorecard.UpperSum(),3}
            ↪ Bonus: {(p.Scorecard.UpperBonus > 0 ? "+35" : "
            ↪ 0 ")}");
        sb.AppendLine($"    Lower:
            ↪ {p.Scorecard.LowerSum(),3}");
    }
}
```

```
        sb.AppendLine($"      Total: {p.Scorecard.GrandTotal,3}
        ↳ Filled: {filled}/13");
        sb.AppendLine();
    }

    _rtbScoreboard.Text = sb.ToString();
}
}
```